

Fix Recommendation Agent 逐行中文注释

文件头部和导入部分 (第1-24行)

```
"""  
Fix Recommendation Agent for generating intelligent suggestions to resolve data quality issues.  
"""
```

翻译: 修复推荐代理, 用于生成智能建议以解决数据质量问题。

```
import logging
```

翻译: 导入日志记录模块, 用于记录程序运行信息。

```
import time
```

翻译: 导入时间模块, 用于测量执行时间。

```
import uuid
```

翻译: 导入UUID模块, 用于生成唯一标识符。

```
from typing import Dict, List, Optional, Any, Tuple
```

翻译: 从typing模块导入类型提示: 字典、列表、可选类型、任意类型、元组。

```
from datetime import datetime
```

翻译: 从datetime模块导入datetime类, 用于处理日期时间。

```
from src.agents.base_agent import BaseAgent
```

翻译: 导入基础代理类, FixRecommendationAgent将继承此类。

```
from src.core.config_manager import ConfigManager
```

翻译: 导入配置管理器, 用于读取配置文件。

```
from src.core.data_models import (  
    FixRecommendation,  
    FixRecommendationResults,  
    Issue,  
    Anomaly,  
    ProfileResults,  
    AnomalyDetectionResults,  
    ColumnMetrics,  
)
```

翻译: 导入数据模型类:

- FixRecommendation: 单个修复建议的数据模型
- FixRecommendationResults: 修复建议结果集合
- Issue: 数据质量问题

- Anomaly: 异常检测结果
- ProfileResults: 数据画像结果
- AnomalyDetectionResults: 异常检测结果集合
- ColumnMetrics: 列指标

```
from src.core.validation_rule_loader import ValidationRuleLoader
```

翻译: 导入验证规则加载器, 用于加载数据验证规则。

```
logger = logging.getLogger(__name__)
```

翻译: 创建日志记录器, 使用当前模块名(name)作为日志名称。

类定义和初始化 (第27-79行)

```
class FixRecommendationAgent(BaseAgent):
```

翻译: 定义修复推荐代理类, 继承自BaseAgent基础类。

```
"""
Agent responsible for generating intelligent fix recommendations for data quality issues.

Analyzes issues and anomalies to suggest:
- Completeness: imputation strategies based on patterns
- Conformity: format corrections based on validation rules
- Uniqueness: duplicate removal strategies
- Anomalies: remediation based on historical patterns
"""
```

翻译: 类文档字符串, 说明代理的职责:

- 负责生成智能修复建议以解决数据质量问题
- 分析问题和异常, 提供建议:
 - 完整性(Completeness): 基于模式的填补策略
 - 一致性(Conformity): 基于验证规则的格式修正
 - 唯一性(Uniqueness): 重复值删除策略
 - 异常(Anomalies): 基于历史模式的修复

```
# Safe auto-fix thresholds based on configuration
SAFE_IMPUTATION_CONFIDENCE = 0.85
SAFE_FORMAT_CORRECTION_CONFIDENCE = 0.90
SAFE_DUPLICATE_REMOVAL_CONFIDENCE = 0.95
```

翻译: 定义安全自动修复的置信度阈值 (类常量) :

- 填补操作的置信度阈值: 0.85 (85%)
- 格式校正的置信度阈值: 0.90 (90%)
- 重复删除的置信度阈值: 0.95 (95%)

```
def __init__(
    self,
    validation_rules: Optional[Dict[str, Any]] = None,
    config: Optional[Dict[str, Any]] = None,
    validation_rule_loader: Optional[ValidationRuleLoader] = None
):
```

翻译: 初始化方法, 参数:

- validation_rules: 可选的验证规则字典, 如果没有提供则从配置加载
- config: 可选的配置字典, 如果没有提供则从配置管理器加载
- validation_rule_loader: 可选的验证规则加载器实例

```
"""
Initialize fix recommendation agent.

Args:
    validation_rules: Custom validation rules. If None, loads from config.
    config: Agent configuration. If None, loads from config manager.
    validation_rule_loader: ValidationRuleLoader instance. If None, creates new one.
"""
```

翻译: 初始化方法的文档字符串, 说明各参数用途。

```
super().__init__("fix_recommendation", config)
```

翻译: 调用父类BaseAgent的初始化方法, 传入代理名称"fix_recommendation"和配置。

```
# Load configuration if not provided
if config is None:
    config_manager = ConfigManager()
    self.config = config_manager.get_agent_config("fix_recommendation")
```

翻译: 如果没有提供配置, 则创建配置管理器并从配置文件加载"fix_recommendation"代理的配置。

```
# Initialize validation rule loader
self.validation_rule_loader = validation_rule_loader or ValidationRuleLoader()
```

翻译: 初始化验证规则加载器, 如果参数中没有提供则创建新实例。

```
# Load validation rules
if validation_rules is None:
    self.validation_rules = self.validation_rule_loader.load_validation_rules()
else:
    self.validation_rules = validation_rules
```

翻译: 加载验证规则, 如果参数中没有提供则通过加载器从配置加载, 否则使用提供的规则。

```
# Agent configuration
self.confidence_threshold = self.config.get("confidence_threshold", 0.75)
self.enable_auto_suggestions = self.config.get("enable_auto_suggestions", True)
self.enable_llm_fixes = self.config.get("enable_llm_fixes", False)
self.llm_config = self.config.get("llm_config", {})
```

翻译: 从配置中读取代理设置:

- confidence_threshold: 置信度阈值, 默认0.75 (75%)
- enable_auto_suggestions: 是否启用自动建议, 默认True
- enable_llm_fixes: 是否启用LLM (大语言模型) 修复, 默认False
- llm_config: LLM配置字典, 默认为空字典

```
logger.info("FixRecommendationAgent initialized")
```

翻译: 记录日志, 表示FixRecommendationAgent初始化完成。

主要执行方法 (第81-182行)

```
def execute(  
    self,  
    profile_results: Optional[ProfileResults] = None,  
    anomaly_results: Optional[AnomalyDetectionResults] = None,  
    issues: Optional[List[Issue]] = None,  
    anomalies: Optional[List[Anomaly]] = None,  
    **kwargs  
) -> FixRecommendationResults:
```

翻译: 主要执行方法, 用于生成修复建议。参数:

- profile_results: 可选的画像结果 (包含问题列表)
 - anomaly_results: 可选的异常检测结果
 - issues: 可选的问题列表 (作为profile_results的替代)
 - anomalies: 可选的异常列表 (作为anomaly_results的替代)
 - **kwargs: 其他关键字参数
- 返回FixRecommendationResults对象。

```
"""  
Generate fix recommendations for detected issues and anomalies.  
  
Args:  
    profile_results: Profile results containing issues  
    anomaly_results: Anomaly detection results  
    issues: Direct list of issues (alternative to profile_results)  
    anomalies: Direct list of anomalies (alternative to anomaly_results)  
  
Returns:  
    FixRecommendationResults containing all recommendations  
"""
```

翻译: 方法文档字符串, 说明该方法为检测到的问题和异常生成修复建议。

```
start_time = time.time()
```

翻译: 记录开始时间, 用于计算执行耗时。

```
# Collect issues and anomalies from various sources  
all_issues = []  
all_anomalies = []
```

翻译: 从不同来源收集问题和异常, 初始化空列表。

```
if profile_results and profile_results.issues:
    all_issues.extend(profile_results.issues)
    dataset_name = profile_results.dataset_profile.dataset_name
    run_id = profile_results.execution_metadata.get("run_id")
```

翻译: 如果提供了profile_results且包含问题列表, 则:

- 将问题添加到all_issues列表
- 从数据集画像中获取数据集名称
- 从执行元数据中获取运行ID

```
elif issues:
    all_issues = issues
    dataset_name = kwargs.get("dataset_name", "unknown")
    run_id = kwargs.get("run_id")
```

翻译: 否则如果直接提供了问题列表, 则:

- 使用直接提供的问题列表
- 从kwargs获取数据集名称, 默认为"unknown"
- 从kwargs获取运行ID

```
else:
    dataset_name = kwargs.get("dataset_name", "unknown")
    run_id = kwargs.get("run_id")
```

翻译: 如果既没有profile_results也没有issues, 则:

- 从kwargs获取数据集名称, 默认为"unknown"
- 从kwargs获取运行ID

```
if anomaly_results and anomaly_results.anomalies:
    all_anomalies.extend(anomaly_results.anomalies)
elif anomalies:
    all_anomalies = anomalies
```

翻译: 处理异常:

- 如果提供了anomaly_results且包含异常列表, 则添加到all_anomalies
- 否则如果直接提供了anomalies列表, 则使用它

```
if not all_issues and not all_anomalies:
    logger.warning("No issues or anomalies provided for recommendation generation")
```

翻译: 如果既没有问题也没有异常, 记录警告日志。

```

return FixRecommendationResults(
    dataset_name=dataset_name or "unknown",
    run_id=run_id,
    recommendations=[],
    execution_metadata={
        "execution_time_seconds": time.time() - start_time,
        "agent_version": "1.0.0",
        "timestamp": datetime.now().isoformat(),
        "issues_analyzed": 0,
        "anomalies_analyzed": 0,
        "total_recommendations": 0,
        "actionable_recommendations": 0
    }
)

```

翻译: 返回空的修复建议结果对象, 包含:

- 数据集名称 (或"unknown")
- 运行ID
- 空的建议列表
- 执行元数据: 执行时间、代理版本、时间戳、分析的问题数 (0)、分析的异常数 (0)、总建议数 (0)、可执行的建议数 (0)

```

logger.info(
    f"Generating fix recommendations for {len(all_issues)} issues "
    f"and {len(all_anomalies)} anomalies"
)

```

翻译: 记录信息日志, 说明正在为多少个问题和异常生成修复建议。

```
recommendations = []
```

翻译: 初始化建议列表为空。

```

# Generate recommendations for issues
if profile_results:
    recommendations.extend(
        self._recommend_for_issues(all_issues, profile_results.dataset_profile)
    )
else:
    recommendations.extend(self._recommend_for_issues_simple(all_issues))

```

翻译: 为问题生成建议:

- 如果有profile_results, 使用完整上下文生成建议
- 否则使用简化方法生成建议

```

# Generate recommendations for anomalies
recommendations.extend(self._recommend_for_anomalies(all_anomalies))

```

翻译: 为异常生成建议并添加到列表中。

```
execution_time = time.time() - start_time
```

翻译: 计算执行时间（当前时间 - 开始时间）。

```
metadata = {
    "execution_time_seconds": execution_time,
    "agent_version": "1.0.0",
    "timestamp": datetime.now().isoformat(),
    "issues_analyzed": len(all_issues),
    "anomalies_analyzed": len(all_anomalies),
    "total_recommendations": len(recommendations),
    "actionable_recommendations": sum(1 for r in recommendations if r.actionable)
}
```

翻译: 构建执行元数据字典，包含：

- 执行时间（秒）
- 代理版本号
- 时间戳（ISO格式）
- 分析的问题数量
- 分析的异常数量
- 总建议数量
- 可执行的建议数量（统计actionable=True的建议）

```
logger.info(
    f"Generated {len(recommendations)} recommendations "
    f"({sum(1 for r in recommendations if r.actionable)} actionable) "
    f"in {execution_time:.2f}s"
)
```

翻译: 记录信息日志，说明生成了多少个建议、其中多少个可执行、耗时多少秒。

```
return FixRecommendationResults(
    dataset_name=dataset_name,
    run_id=run_id,
    recommendations=recommendations,
    execution_metadata=metadata
)
```

翻译: 返回修复建议结果对象，包含数据集名称、运行ID、建议列表和执行元数据。

问题建议生成方法 (第184-236行)

```
def _recommend_for_issues(
    self,
    issues: List[Issue],
    dataset_profile
) -> List[FixRecommendation]:
```

翻译: 为问题列表生成建议（带完整画像上下文）。参数：

- issues: 问题列表
 - dataset_profile: 数据集画像对象
- 返回修复建议列表。

```
"""Generate recommendations for a list of issues with full profile context."""
```

翻译: 方法文档: 使用完整画像上下文为问题列表生成建议。

```
recommendations = []

for issue in issues:
    rec = self._recommend_for_issue(issue, dataset_profile)
    if rec:
        recommendations.append(rec)

return recommendations
```

翻译:

- 初始化建议列表
- 遍历每个问题, 为单个问题生成建议
- 如果生成了建议, 则添加到列表
- 返回建议列表

```
def _recommend_for_issues_simple(self, issues: List[Issue]) -> List[FixRecommendation]:
```

翻译: 为问题列表生成建议 (无完整上下文) 。

```
"""Generate recommendations for a list of issues without full profile context."""
```

翻译: 方法文档: 不使用完整画像上下文为问题列表生成建议。

```
recommendations = []

for issue in issues:
    rec = self._recommend_for_issue_simple(issue)
    if rec:
        recommendations.append(rec)

return recommendations
```

翻译: 类似上面, 但使用简化方法为单个问题生成建议。

```
def _recommend_for_issue(
    self,
    issue: Issue,
    dataset_profile
) -> Optional[FixRecommendation]:
```

翻译: 为单个问题生成建议 (带完整上下文) 。返回可选的修复建议。

```
"""Generate a recommendation for a single issue."""
```

翻译: 方法文档: 为单个问题生成建议。

```
column_metrics = dataset_profile.column_metrics.get(issue.column_name)
```

翻译: 从数据集画像中获取该问题列的指标。

```

if issue.issue_type == "completeness":
    return self._recommend_completeness_fix(issue, column_metrics, dataset_profile)
elif issue.issue_type == "conformity":
    return self._recommend_conformity_fix(issue, column_metrics)
elif issue.issue_type == "uniqueness":
    return self._recommend_uniqueness_fix(issue, column_metrics)

return None

```

翻译: 根据问题类型调用相应的修复建议方法:

- "completeness" (完整性): 调用完整性修复建议方法
- "conformity" (一致性): 调用一致性修复建议方法
- "uniqueness" (唯一性): 调用唯一性修复建议方法
- 其他类型返回None

```

def _recommend_for_issue_simple(self, issue: Issue) -> Optional[FixRecommendation]:

```

翻译: 为单个问题生成建议 (无完整上下文) 。

```

"""Generate a recommendation for a single issue without full context."""

```

翻译: 方法文档: 不使用完整上下文为单个问题生成建议。

```

if issue.issue_type == "completeness":
    return self._recommend_completeness_fix_simple(issue)
elif issue.issue_type == "conformity":
    return self._recommend_conformity_fix_simple(issue)
elif issue.issue_type == "uniqueness":
    return self._recommend_uniqueness_fix_simple(issue)

return None

```

翻译: 类似上面, 但使用简化的修复建议方法。

完整性修复建议 (第238-330行)

```

def _recommend_completeness_fix(
    self,
    issue: Issue,
    column_metrics: Optional[ColumnMetrics],
    dataset_profile
) -> Optional[FixRecommendation]:

```

翻译: 为完整性问题生成修复建议。参数包括问题、列指标、数据集画像。

```

"""Recommend fix for completeness issues."""

```

翻译: 方法文档: 为完整性问题推荐修复方案。

```

if not column_metrics:
    return self._recommend_completeness_fix_simple(issue)

```

翻译: 如果没有列指标, 使用简化方法。

```
# Analyze column patterns to suggest imputation
suggested_value = None
confidence = 0.5
actionable = False
strategy = "flag_for_review"
rationale = "" # Initialize rationale
```

翻译: 初始化变量:

- suggested_value: 建议值 (None)
- confidence: 置信度 (0.5, 即50%)
- actionable: 是否可自动执行 (False)
- strategy: 策略 ("flag_for_review", 标记为需要审查)
- rationale: 理由 (空字符串)

```
# Strategy 1: Use most common value if sufficient data
if column_metrics.most_common_values:
    top_value, top_count = column_metrics.most_common_values[0]
    coverage = (top_count / column_metrics.total_count) * 100

    if coverage > 20: # If >20% of data has this value
        suggested_value = top_value
        confidence = min(0.7, coverage / 100)
        strategy = "mode_imputation"
        rationale = f"Mode value '{top_value}' appears in {coverage:.1f}% of records"
```

翻译: 策略1: 如果数据充分, 使用最常见的值 (众数):

- 获取最常见的值和出现次数
- 计算覆盖率 (百分比)
- 如果覆盖率>20%, 则:
 - 建议值设为该最常见值
 - 置信度设为最小值 (0.7, 覆盖率/100)
 - 策略设为"mode_imputation" (众数填补)
 - 理由说明该值在记录中的覆盖率

```
# Strategy 2: Use default values for specific columns
if not suggested_value:
    default_values = self._get_column_defaults(issue.column_name)
    if default_values:
        suggested_value = default_values
        confidence = 0.6
        strategy = "default_imputation"
        rationale = f"Using standard default for {issue.column_name}"
```

翻译: 策略2: 如果没有建议值, 尝试使用特定列的默认值:

- 获取该列的默认值
- 如果有默认值, 则:
 - 建议值设为默认值

- 置信度设为0.6（60%）
- 策略设为"default_imputation"（默认值填补）
- 理由说明使用标准默认值

```
# Determine if safe for auto-fix
if confidence >= self.SAFE_IMPUTATION_CONFIDENCE:
    actionable = True
    estimated_impact = "low" if issue.percentage < 5 else "medium"
else:
    estimated_impact = "high"
    if rationale:
        rationale += " - requires review due to low confidence"
```

翻译: 判断是否安全可自动修复:

- 如果置信度>=安全填补阈值（0.85），则：
 - actionable设为True（可自动执行）
 - 如果问题百分比<5%，影响估计为"low"，否则为"medium"
- 否则：
 - 影响估计为"high"
 - 如果已有理由，追加说明需要审查（因为置信度低）

```
if not rationale:
    rationale = f"Missing values detected in {issue.column_name} - manual review recommended"
```

翻译: 如果没有理由，设置默认理由：检测到缺失值，建议人工审查。

```
return FixRecommendation(
    recommendation_id=str(uuid.uuid4()),
    issue_id=issue.issue_id,
    dataset_name=dataset_profile.dataset_name,
    column_name=issue.column_name,
    issue_type=issue.issue_type,
    fix_strategy=strategy,
    fix_description=f"Fill {issue.count} missing values in {issue.column_name}",
    confidence_score=confidence,
    estimated_impact=estimated_impact,
    actionable=actionable,
    suggested_value=suggested_value,
    rationale=rationale,
    prerequisites=["backup_dataset", "validate_suggested_value"],
    risk_assessment="Low risk: value imputation based on data patterns" if actionable else "Medium
risk: requires domain expert review"
)
```

翻译: 返回修复建议对象，包含：

- recommendation_id: 建议唯一ID（UUID）
- issue_id: 关联的问题ID
- dataset_name: 数据集名称
- column_name: 列名

- issue_type: 问题类型
- fix_strategy: 修复策略
- fix_description: 修复描述 (填充多少个缺失值)
- confidence_score: 置信度分数
- estimated_impact: 估计影响 (low/medium/high)
- actionable: 是否可自动执行
- suggested_value: 建议值
- rationale: 理由
- prerequisites: 前置条件 (备份数据集、验证建议值)
- risk_assessment: 风险评估 (如果可自动执行则风险低, 否则需要领域专家审查)

```
def _recommend_completeness_fix_simple(self, issue: Issue) -> Optional[FixRecommendation]:
```

翻译: 为完整性问题生成修复建议 (简化版, 无完整上下文)。

```
"""Recommend fix for completeness issues without full context."""
```

翻译: 方法文档: 不使用完整上下文为完整性问题推荐修复方案。

```
suggested_value = None
strategy = "flag_for_review"

# Try to use defaults
default_values = self._get_column_defaults(issue.column_name)
if default_values:
    suggested_value = default_values
    strategy = "default_imputation"
```

翻译:

- 初始化建议值和策略
- 尝试使用默认值
- 如果有默认值, 则设为建议值, 策略设为"default_imputation"

```
return FixRecommendation(
    recommendation_id=str(uuid.uuid4()),
    issue_id=issue.issue_id,
    dataset_name="unknown",
    column_name=issue.column_name,
    issue_type=issue.issue_type,
    fix_strategy=strategy,
    fix_description=f"Investigate and fill {issue.count} missing values in {issue.column_name}",
    confidence_score=0.6,
    estimated_impact="medium",
    actionable=False,
    suggested_value=suggested_value,
    rationale=f"Manual review required for missing values in {issue.column_name}",
    prerequisites=["backup_dataset", "domain_expert_review"],
    risk_assessment="Manual intervention required"
)
```

翻译: 返回修复建议对象 (简化版), 置信度固定0.6, 不可自动执行, 需要人工审查。

一致性修复建议 (第332-406行)

```
def _recommend_conformity_fix(
    self,
    issue: Issue,
    column_metrics: Optional[ColumnMetrics]
) -> Optional[FixRecommendation]:
```

翻译: 为一致性问题生成修复建议。

```
"""Recommend fix for conformity issues."""
```

翻译: 方法文档: 为一致性问题推荐修复方案。

```
if not column_metrics or not issue.examples:
    return self._recommend_conformity_fix_simple(issue)
```

翻译: 如果没有列指标或问题示例, 使用简化方法。

```
# Analyze violation examples
violations = issue.examples[:5]
strategy = "format_correction"
confidence = 0.7
actionable = False
```

翻译:

- 获取前5个违反规则的示例
- 策略设为"format_correction" (格式修正)
- 置信度初始化为0.7 (70%)
- 不可自动执行

```
# Try to identify common patterns in violations
rule = self.validation_rules.get(issue.column_name)
if not rule:
    return self._recommend_conformity_fix_simple(issue)
```

翻译:

- 尝试识别违反规则中的常见模式
- 获取该列的验证规则
- 如果没有规则, 使用简化方法

```
# Analyze violations based on rule type
if rule.rule_type == "regex":
    suggested_fixes = self._analyze_regex_violations(violations, rule)
elif rule.rule_type == "length":
    suggested_fixes = self._analyze_length_violations(violations, rule)
elif rule.rule_type == "enum":
    suggested_fixes = self._analyze_enum_violations(violations, rule)
else:
    suggested_fixes = []
```

翻译: 根据规则类型分析违反情况:

- "regex" (正则表达式) : 分析正则表达式违反
- "length" (长度) : 分析长度违反
- "enum" (枚举) : 分析枚举违反
- 其他类型: 空列表

```
rationale = f"{len(violations)} format violations detected in {issue.column_name}"
if suggested_fixes:
    rationale += f". Common fixes: {' '.join(suggested_fixes[:3])}"
confidence = 0.8
```

翻译:

- 构建理由: 说明检测到多少个格式违反
- 如果有建议修复, 追加常见修复方法 (前3个), 并将置信度提升至0.8

```
# Determine if safe for auto-fix
if confidence >= self.SAFE_FORMAT_CORRECTION_CONFIDENCE and len(suggested_fixes) > 0:
    actionable = True
    estimated_impact = "low"
else:
    estimated_impact = "medium"
```

翻译: 判断是否安全可自动修复:

- 如果置信度>=安全格式校正阈值 (0.90) 且有建议修复, 则:
 - 可自动执行, 影响估计为"low"
- 否则影响估计为"medium"

```
return FixRecommendation(
    recommendation_id=str(uuid.uuid4()),
    issue_id=issue.issue_id,
    dataset_name="unknown",
    column_name=issue.column_name,
    issue_type=issue.issue_type,
    fix_strategy=strategy,
    fix_description=f"Correct {issue.count} format violations in {issue.column_name}",
    confidence_score=confidence,
    estimated_impact=estimated_impact,
    actionable=actionable,
    rationale=rationale,
    prerequisites=["backup_dataset", "validate_format_corrections"],
    risk_assessment="Data transformation required"
)
```

翻译: 返回修复建议对象, 包含格式修正相关信息。

```
def _recommend_conformity_fix_simple(self, issue: Issue) -> Optional[FixRecommendation]:
```

翻译: 为一致性问题生成修复建议 (简化版) 。

```
"""Recommend fix for conformity issues without full context."""
```

翻译: 方法文档: 不使用完整上下文为一致性问题推荐修复方案。

```

return FixRecommendation(
    recommendation_id=uuid.uuid4(),
    issue_id=issue.issue_id,
    dataset_name="unknown",
    column_name=issue.column_name,
    issue_type=issue.issue_type,
    fix_strategy="format_correction",
    fix_description=f"Investigate and correct {issue.count} format violations in {issue.column_name}",
    confidence_score=0.6,
    estimated_impact="medium",
    actionable=False,
    rationale="Format violations detected - manual review required",
    prerequisites=["backup_dataset", "validate_format_corrections"],
    risk_assessment="Manual intervention required"
)

```

翻译: 返回修复建议对象（简化版），需要人工审查。

唯一性修复建议 (第408-454行)

```

def _recommend_uniqueness_fix(
    self,
    issue: Issue,
    column_metrics: Optional[ColumnMetrics]
) -> Optional[FixRecommendation]:

```

翻译: 为唯一性问题生成修复建议。

```

"""Recommend fix for uniqueness issues (duplicates)."""

```

翻译: 方法文档: 为唯一性问题（重复值）推荐修复方案。

```

if not column_metrics or issue.column_name != "NPI":
    return self._recommend_uniqueness_fix_simple(issue)

```

翻译: 如果没有列指标或不是NPI列，使用简化方法。

```

# For NPI duplicates, always flag for review as it's critical
confidence = 0.95
actionable = False
strategy = "remove_duplicates"

```

翻译: 对于NPI重复值（关键字段），总是标记为需要审查：

- 置信度设为0.95（95%）
- 不可自动执行（需要人工审查）
- 策略设为"remove_duplicates"（删除重复）

```

return FixRecommendation(
    recommendation_id=uuid.uuid4(),
    issue_id=issue.issue_id,
    dataset_name="unknown",
    column_name=issue.column_name,
    issue_type=issue.issue_type,
    fix_strategy=strategy,
    fix_description=f"Investigate and remove {issue.count} duplicate NPI values",
    confidence_score=confidence,
    estimated_impact="high",
    actionable=actionable,
    rationale=f"Critical: NPI must be unique. {issue.count} duplicates detected",
    prerequisites=["backup_dataset", "investigate_duplicate_source"],
    risk_assessment="HIGH RISK: Duplicate NPIs may indicate data integrity issues"
)

```

翻译: 返回修复建议对象, 说明NPI必须唯一, 检测到重复, 影响高, 需要调查重复来源, 风险高。

```
def _recommend_uniqueness_fix_simple(self, issue: Issue) -> Optional[FixRecommendation]:
```

翻译: 为唯一性问题生成修复建议 (简化版)。

```
"""Recommend fix for uniqueness issues without full context."""
```

翻译: 方法文档: 不使用完整上下文为唯一性问题推荐修复方案。

```

return FixRecommendation(
    recommendation_id=uuid.uuid4(),
    issue_id=issue.issue_id,
    dataset_name="unknown",
    column_name=issue.column_name,
    issue_type=issue.issue_type,
    fix_strategy="remove_duplicates",
    fix_description=f"Investigate and remove {issue.count} duplicate values in {issue.column_name}",
    confidence_score=0.8,
    estimated_impact="high",
    actionable=False,
    rationale=f"Duplicates detected - requires investigation",
    prerequisites=["backup_dataset", "investigate_duplicate_source"],
    risk_assessment="Manual investigation required"
)

```

翻译: 返回修复建议对象 (简化版), 需要人工调查。

异常修复建议 (第456-500行)

```
def _recommend_for_anomalies(self, anomalies: List[Anomaly]) -> List[FixRecommendation]:
```

翻译: 为异常列表生成修复建议。

```
"""Generate recommendations for anomaly issues."""
```

翻译: 方法文档: 为异常问题生成建议。

```
recommendations = []

for anomaly in anomalies:
    rec = self._recommend_for_anomaly(anomaly)
    if rec:
        recommendations.append(rec)

return recommendations
```

翻译: 遍历每个异常, 为单个异常生成建议, 收集到列表中返回。

```
def _recommend_for_anomaly(self, anomaly: Anomaly) -> Optional[FixRecommendation]:
```

翻译: 为单个异常生成修复建议。

```
"""Generate a recommendation for a single anomaly."""
```

翻译: 方法文档: 为单个异常生成建议。

```
# Anomalies typically indicate systemic issues requiring investigation
strategy = "investigate_root_cause"
confidence = 0.7 if anomaly.severity == "high" else 0.5
actionable = False
```

翻译:

- 异常通常表示需要调查的系统性问题
- 策略设为"investigate_root_cause" (调查根本原因)
- 如果严重程度为"high", 置信度为0.7, 否则为0.5
- 不可自动执行

```
# Determine recommendation based on anomaly direction
if anomaly.delta < 0: # Quality degraded
    description = f"{anomaly.metric} dropped {abs(anomaly.delta):.1f}% from baseline"
    rationale = f"Degradation detected: {anomaly.metric} fell from {anomaly.baseline_value:.1f}% to {anomaly.current_value:.1f}%"
else: # Quality improved unexpectedly
    description = f"{anomaly.metric} increased {anomaly.delta:.1f}% from baseline"
    rationale = f"Unexpected improvement: {anomaly.metric} rose from {anomaly.baseline_value:.1f}% to {anomaly.current_value:.1f}%"
```

翻译: 根据异常方向确定建议:

- 如果delta<0 (质量下降): 描述和理由说明指标下降了多少
- 否则 (质量意外提升): 描述和理由说明指标上升了多少

```
# Adjust confidence based on severity and z-score
if anomaly.severity == "high" and anomaly.z_score > 3:
    confidence = 0.9
```

翻译: 根据严重程度和z分数调整置信度:

- 如果严重程度为"high"且z分数>3, 置信度提升至0.9

```

return FixRecommendation(
    recommendation_id=str(uuid.uuid4()),
    issue_id=anomaly.anomaly_id,
    dataset_name=anomaly.dataset_name,
    column_name=anomaly.column_name,
    issue_type="anomaly",
    fix_strategy=strategy,
    fix_description=description,
    confidence_score=confidence,
    estimated_impact=anomaly.severity,
    actionable=actionable,
    rationale=rationale,
    prerequisites=["backup_dataset", "investigate_upstream_source", "check_data_ingestion_process"],
    risk_assessment=f"Systematic issue detected (z-score: {anomaly.z_score:.2f}, severity: {anomaly.severity})"
)

```

翻译: 返回修复建议对象, 说明检测到系统性问题, 需要调查上游数据源和数据摄取过程, 风险评估包含z分数和严重程度。

辅助方法 (第502-534行)

```
def _get_column_defaults(self, column_name: str) -> Optional[Any]:
```

翻译: 获取特定列的默认值。

```
"""Get default values for specific columns."""
```

翻译: 方法文档: 获取特定列的默认值。

```

defaults = {
    "Entity Type Code": "1",
    "Last Update Date": datetime.now().strftime("%m/%d/%Y"),
}
return defaults.get(column_name)

```

翻译:

- 定义默认值字典:
 - "Entity Type Code" (实体类型代码) : "1"
 - "Last Update Date" (最后更新日期) : 当前日期的格式化字符串 (MM/DD/YYYY)
- 返回该列的默认值 (如果没有则返回None)

```
def _analyze_regex_violations(self, violations: List[str], rule) -> List[str]:
```

翻译: 分析正则表达式违反情况, 建议修复方法。

```
"""Analyze regex violations to suggest fixes."""
```

翻译: 方法文档: 分析正则表达式违反, 建议修复方法。

```
# For now, return general advice
return ["Check regex pattern", "Verify data source", "Apply format transformation"]
```

翻译: 目前返回通用建议: 检查正则表达式模式、验证数据源、应用格式转换。

```
def _analyze_length_violations(self, violations: List[str], rule) -> List[str]:
```

翻译: 分析长度违反情况, 建议修复方法。

```
"""Analyze length violations to suggest fixes."""
```

翻译: 方法文档: 分析长度违反, 建议修复方法。

```
suggestions = []
min_len = rule.rule_config.get("min", 0)
max_len = rule.rule_config.get("max", float('inf'))
```

翻译:

- 初始化建议列表
- 从规则配置获取最小长度 (默认0) 和最大长度 (默认无穷大)

```
for violation in violations:
    vlen = len(str(violation))
    if vlen < min_len:
        suggestions.append("Truncate values that are too short")
    if vlen > max_len:
        suggestions.append("Truncate values that are too long")

return list(set(suggestions))
```

翻译:

- 遍历每个违反值
- 计算长度
- 如果长度<最小长度, 添加建议"截断过短的值"
- 如果长度>最大长度, 添加建议"截断过长的值"
- 返回去重后的建议列表

```
def _analyze_enum_violations(self, violations: List[str], rule) -> List[str]:
```

翻译: 分析枚举违反情况, 建议修复方法。

```
"""Analyze enum violations to suggest fixes."""
```

翻译: 方法文档: 分析枚举违反, 建议修复方法。

```
allowed = rule.rule_config.get("values", [])
return [f"Ensure value is one of: {' '.join(allowed)}"]
```

翻译:

- 从规则配置获取允许的值列表
 - 返回建议: 确保值在允许的枚举值中
-

总结

这个代理类的核心功能是：

1. 接收问题和异常 - 从画像结果或异常检测结果中获取数据质量问题
2. 分析问题类型 - 根据问题类型（完整性、一致性、唯一性、异常）选择相应的修复策略
3. 生成智能建议 - 基于数据模式、验证规则和历史数据生成修复建议
4. 评估可执行性 - 根据置信度判断建议是否可以自动执行
5. 返回建议结果 - 包含详细的修复策略、置信度、风险评估等信息